INSTITUTO POLITÉCTICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

REPORTE DE PRACTICAS

Presenta:

Contreras Sandoval Yuliana Sarai
Hernandez Mendez Jesus Israel

Secuencia:

3CM32

Materia:

Abstraccion y uso de datos

ÍNDICE

1, Introducción 2

2. Programas 3-9

3. Lista de errores 10

4. Conclusiones 11

INTRODUCCIÓN

En el presente trabajo se analiza el uso de distintos conceptos fundamentales de la programación, a través del desarrollo de varios programas que permiten comprender su aplicación práctica. A lo largo del documento se abordan temas como estructuras de datos, manejo de arreglos, uso de punteros, matrices y distintos métodos de ordenamiento, con el propósito de reforzar el aprendizaje tanto en Programación Estructurada como en Programación Orientada a Objetos.

Cada programa fue diseñado con la finalidad de ejemplificar el funcionamiento de un concepto específico, mostrando cómo se construyen desde cero y cómo se aplican en la solución de problemas comunes. Asimismo, se describen las operaciones básicas involucradas, como la captura de datos, su procesamiento y la presentación de resultados.

En particular, se trabajan estructuras como pilas, colas y listas, las cuales permiten organizar la información de diferentes formas. Las pilas operan bajo el principio LIFO (último en entrar, primero en salir), mientras que las colas siguen el modelo FIFO (primero en entrar, primero en salir). Por otro lado, las listas ofrecen mayor flexibilidad al permitir el acceso y modificación de elementos en distintas posiciones.

Además, se incluyen programas que utilizan arreglos y punteros para optimizar el manejo de datos, así como el uso de matrices para realizar operaciones matemáticas. También se exploran diferentes algoritmos de ordenamiento, como burbuja, MergeSort y QuickSort, aplicados tanto de forma directa como indirecta.

En general, este conjunto de programas permite entender la relación entre los distintos elementos de la programación, facilitando el desarrollo de soluciones más completas y eficientes. De esta manera, se fortalece la lógica de programación y la capacidad de resolver problemas mediante el uso adecuado de herramientas y técnicas.

PROGRAMAS

Programa 10

Descripción del funcionamiento del programa: Este programa permite introducir cinco números mediante el uso de un puntero hacia un arreglo. Después de capturarlos, se realizan distintos cálculos como la suma total, el promedio, el valor mayor y el menor. Para ello se utiliza una clase con funciones encargadas de procesar los datos y mostrar los resultados.

```
G:\Dan progras abstraccion\D

Ingresa 5 numeros con espacio: 12 45 3 56 7
Suma: 123
Media: 24.6
Maximo: 56
Minimo: 3

--------------------------------
Process exited after 9.21 seconds with return value 0
Presione una tecla para continuar . . .
```

Programa 11

Descripción del funcionamiento del programa: Este programa trabaja con matrices y ejecuta dos operaciones principales: multiplicación de una matriz por un valor constante y multiplicación entre dos matrices (A × B). Se implementa mediante una clase que contiene métodos para ingresar los datos, realizar las operaciones correspondientes y desplegar los resultados.

```
G:\Dan progras abstraccion\D

Ingresa 5 numeros con espacio: 12 45 3 56 7
Suma: 123
Media: 24.6
Maximo: 56
Minimo: 3

--------------------------------
Process exited after 9.21 seconds with return value 0
Presione una tecla para continuar . . .
```

Programa 12

El usuario indica cuántas personas desea registrar. Posteriormente, captura información como nombre, apellidos, edad y género. Finalmente, se muestra una lista únicamente con aquellas personas que cuentan con automóvil.

```
¿Cuántas personas deseas registrar? 3

--- Persona #1 ---
Nombre: Daniela
Apellido paterno: Trejo
Apellido materno: Cosio
Género (M/F): F
Edad: 19
¿Tiene coche? (s/n): N

--- Persona #2 ---
Nombre: Jorge
Apellido paterno: Checa
Apellido materno: Ramirez
Género (M/F): M
Edad: 34
¿Tiene coche? (s/n): S
Modelo del coche: Toyota
Precio: 180000
Año: 2005

--- Persona #3 ---
Nombre: Iana
Apellido paterno: Fernandez
Apellido materno: Monse
Género (M/F): F
Edad: 33
¿Tiene coche? (s/n): s
Modelo del coche: Tsuru
Precio: 120500
Año: 2003
```

Programa 13

Este programa funciona de manera similar al anterior, pero utilizando punteros para administrar la información capturada.

Programa 14

El usuario selecciona una opción del menú (9 o 10) y posteriormente introduce un número entero. Dependiendo de la elección, el programa calcula y muestra el factorial o la sucesión de Fibonacci.

```
----------------------------------------
                  Menu
----------------------------------------
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Raiz Cuadrada
6. Seno
7. Coseno
8. Potencia
9. Fibonacci
10. Factorial
Opcion: 9
Numero 1: 25
Fibonacci es 75025
```

Programa 15

El usuario elige una opción y después ingresa un número entero. El programa genera representaciones relacionadas con el triángulo de Sierpinski o el conjunto de Cantor.

```
----------------------------------------
                  Menu
----------------------------------------
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Raiz Cuadrada
6. Seno
7. Coseno
8. Potencia
9. Fibonacci
10. Factorial
Opcion: 9
Numero 1: 25
Fibonacci es 75025
```

Programa 16 (Ordenamiento Burbuja)

Se solicita el tamaño del arreglo y el tipo de datos a ordenar. Después, se aplica el método de ordenamiento burbuja para organizar los elementos.

Programa 17 (MergeSort)

El programa solicita el tamaño del arreglo y el tipo de datos. Posteriormente, aplica el algoritmo MergeSort para ordenar la información.

Programa 18 (QuickSort)

Se implementa el algoritmo QuickSort haciendo uso de punteros, permitiendo ordenar los datos de manera indirecta.

Programa 19 (Burbuja indirecto)

Se aplica el método burbuja utilizando punteros, manteniendo el mismo proceso pero de forma indirecta.

Programa 20 (Merge indirecto)

Se utiliza el algoritmo MergeSort con referencias indirectas mediante punteros para ordenar los datos.

Programa 21 (QuickSort indirecto)

Se implementa QuickSort con punteros, permitiendo ordenar diferentes tipos de datos de manera eficiente.

```
----------------------------------------
                Menu
----------------------------------------
1. Sumar
2. Restar
3. Multiplicar
4. Dividir
5. Raiz Cuadrada
6. Seno
7. Coseno
8. Potencia
9. Fibonacci
10. Factorial
Opcion: 9
Numero 1: 25
Fibonacci es 75025
```

```
Ingrese el tamanio del arreglo: 10
Tipo de datos:
1. Enteros
2. Caracteres
3. Personas
Seleccione: 2
Caracter [1]: a
Caracter [2]: h
Caracter [3]: j
Caracter [4]: k
Caracter [5]: m
Caracter [6]: l
Caracter [7]: q
Caracter [8]: t
Caracter [9]: c
Caracter [10]: b

Original (sin ordenar): a h j k m l q t c b
Ordenado (indirecto): a b c h j k l m q t
```

```
Ingrese el tamanio del arreglo: 3
Tipo de datos:
1. Enteros
2. Caracteres
3. Personas
Seleccione: 3
Persona [1]
  Nombre: Daniel
  Edad: 32
Persona [2]
  Nombre: Gabriela
  Edad: 2
Persona [3]
  Nombre: Sonia
  Edad: 90

Original (sin ordenar): Daniel (32) Gabriela (2) Sonia (90)
Ordenado (indirecto): Gabriela (2) Daniel (32) Sonia (90)
```

PROGRAMAS DE PILAS. COLAS Y LISTAS

Pilas

Descripción: En este apartado se implementa la estructura de datos pila, la cual trabaja bajo el principio LIFO (último en entrar, primero en salir). El usuario puede ingresar datos, eliminarlos y consultar el elemento en la cima, así como verificar si la estructura se encuentra vacía.

En los programas de librería de pila, lista o cola hechos en C++. El usuario debe ingresar números que se van insertando cada vez que escoja la opción 1. La opción 4 Muestra el contenido de números que ha insertado el usuario.

La opción 5 sirve para identificar si hemos ingresado algún numero. Si ingresamos uno o varios números nos dirá que no esta vacia. En cambio si no hay nada lo dira El tamaño indica cuantos números hemos ingresado a la pila, lista o cola.

La cima es el ultimo dato que ingreso el usuario, ya que las pilas tienen estructura lifo lo que tarda en entrar es lo primero en salir.

El frente es el primer dato o numero que ingreso el usuario, ya que las cola tienen estructura fifo lo primero que entra es el primero en salir.

```
--- Menu Pila ---
1. Insertar
2. Eliminar
3. Ver cima
4. Mostrar contenido
5. Verificar si esta vacia
6. Mostrar tamano
7. Salir
Opcion: 4
Contenido de la pila (de arriba hacia abajo):
[0] -> 10
[1] -> 9
```

Colas

Descripción: Se desarrolla la estructura de cola, que sigue el modelo FIFO (primero en entrar, primero en salir). Permite agregar elementos, eliminarlos en orden de llegada y visualizar el elemento al frente.

```
--- Menu Cola ---
1. Insertar (enqueue)
2. Eliminar (dequeue)
3. Ver frente
4. Mostrar contenido
5. Verificar si esta vacia
6. Mostrar tamano
7. Salir
Opcion: 3
Elemento en el frente: 9
```

Listas

Descripción: Se implementa una lista que permite almacenar datos de manera flexible. El usuario puede insertar, eliminar, buscar elementos y conocer su posición dentro de la lista.

```
--- Menu Lista  ---
1. Insertar
2. Eliminar
3. Buscar
4. Mostrar contenido
5. Verificar si esta vacia
6. Mostrar tamano
7. Ordenar lista
8. Salir
Opcion: 3
Elemento a buscar: 5
No encontrado.
```

Pilas, Colas y Listas Dinámicas

Descripción: En esta sección, las estructuras se trabajan de forma dinámica mediante el uso de memoria dinámica. Esto permite manejar los datos de manera más eficiente sin limitar el tamaño de las estructuras.

Uso de Librerías

Descripción: Se emplean librerías en C++ para facilitar la implementación de pilas, colas y listas. El usuario interactúa con menús que permiten insertar datos, visualizarlos, verificar si la estructura está vacía y conocer su tamaño.

Funciones adicionales

Descripción: Se incluyen funciones que permiten mostrar el contenido de las estructuras, verificar si están vacías y conocer características como el tamaño, el elemento inicial (frente) y el elemento final (cima).

LISTA DE ERRORES

Durante la elaboración de los programas se presentaron diversos errores que fueron corregidos conforme se avanzaba en el desarrollo del trabajo. A continuación, se mencionan los más relevantes:

Se realizaron ajustes en programas que tenían datos predefinidos, permitiendo que el usuario pudiera ingresar la información manualmente para un mejor funcionamiento.

En el programa de matrices (Programa 11), se corrigió el método de multiplicación, ya que inicialmente se realizaba de forma incorrecta. Se implementó el procedimiento adecuado siguiendo las reglas del álgebra lineal, multiplicando filas por columnas.

En el programa 6 (de reportes anteriores), se corrigió la operación de división para mostrar correctamente el cociente y el residuo.

Se agregó validación en algunos programas para evitar el uso de valores negativos en operaciones donde no eran permitidos.

Se mejoró el código de listas incluyendo funciones de ordenamiento que no estaban consideradas inicialmente.

Algunos programas que estaban desarrollados en otros entornos (como NetBeans) fueron adaptados correctamente al lenguaje C++.

Se corrigieron errores en el manejo de punteros para evitar fallas en la ejecución de los programas, especialmente en las versiones indirectas de los algoritmos de ordenamiento.

CONCLUSIONES

En el desarrollo de este trabajo se logró comprender de manera práctica diferentes conceptos fundamentales de la programación mediante la implementación de diversos programas. A través de ellos, se reforzaron temas importantes como el manejo de arreglos, punteros, matrices y el uso de algoritmos de ordenamiento, lo cual permitió mejorar la lógica de programación.

Durante el primer parcial, se trabajaron programas enfocados en la resolución de problemas mediante cálculos, estructuras de control y algoritmos, destacando el uso de métodos como burbuja, MergeSort y QuickSort, los cuales son esenciales para la organización eficiente de datos.

Por otro lado, en el segundo parcial se profundizó en el uso de estructuras de datos como pilas, colas y listas, tanto de forma estática como dinámica. Esto permitió entender cómo se gestionan los datos en memoria y la importancia de elegir la estructura adecuada según el tipo de problema a resolver.

En general, este conjunto de prácticas contribuyó a desarrollar habilidades necesarias para la programación, como el análisis, la organización de información y la implementación de soluciones eficientes. Además, permitió comprender la relación entre diferentes conceptos y su aplicación en situaciones reales.

Finalmente, este trabajo fortalece las bases necesarias para continuar con el aprendizaje en el área de programación, facilitando el desarrollo de programas más complejos y estructurados en el futuro.